

Chapter 1

Identificación de Escenarios

En esta sección se describen los escenarios principales del sistema académico para la enseñanza de programación en Python. Cada escenario representa una instancia concreta de los casos de uso identificados, mostrando la secuencia de interacciones entre actores específicos.

Escenario 1 — Inicio de sesión

Nombre del escenario	inicioSesion
Instancias de actores participantes	maria:Estudiante
Flujo de eventos	1. María accede a la URL de la plataforma desde su navegador e ingresa su correo institucional y contraseña en el formulario de autenticación. 2. El sistema verifica las credenciales contra la base de datos de usuarios registrados. 3. Las credenciales son válidas, por lo que el sistema identifica el rol de María como <i>Estudiante</i> y la redirige a su dashboard personalizado. 4. María visualiza sus cursos activos y la lista de asignaciones pendientes ordenadas por fecha de entrega.

Table 1.1: Escenario inicioSesion para el caso de uso *Autenticar Usuario*.

Escenario 2 — Visualización del dashboard estudiantil

Nombre del escenario	visualizarDashboard
Instancias de actores participantes	carlos:Estudiante
Flujo de eventos	1. Carlos inicia sesión en la plataforma con sus credenciales institucionales. 2. El sistema carga el dashboard de Carlos mostrando la lista de cursos en los que está matriculado. 3. Carlos selecciona el curso <i>Introducción a la Programación</i> para ver sus asignaciones. 4. El sistema despliega las asignaciones del curso, indicando para cada una: título, descripción breve, fecha límite y estado (pendiente, en progreso, entregado). 5. Carlos identifica una asignación pendiente y hace clic en ella para acceder a su detalle y comenzar a trabajar.

Table 1.2: Escenario visualizarDashboard para el caso de uso *Ver Dashboard*.

Escenario 3 — Creación y edición de archivo en la IDE

Nombre del escenario	crearArchivoIDE
Instancias de actores participantes	ana:Estudiante, modCripto:ModuloValidacionCriptografica
Flujo de eventos	1. Ana selecciona una asignación desde su dashboard y hace clic en "Abrir IDE". 2. La plataforma abre la IDE especializada de Python asociada a esa asignación dentro del entorno controlado. 3. Ana crea un nuevo archivo <code>.py</code> utilizando la opción de la IDE. El módulo criptográfico genera el hash inicial del archivo vacío y registra su firma como válida. 4. Ana escribe su solución directamente en la IDE. Las opciones de copiar y pegar están deshabilitadas dentro del editor. 5. Al guardar, la plataforma solicita al módulo criptográfico actualizar la firma del archivo con el nuevo contenido. 6. El módulo actualiza el hash y lo almacena; la IDE muestra un indicador de integridad confirmada en la barra de estado.

Table 1.3: Escenario crearArchivoIDE para el caso de uso *Editar Archivo en IDE*.

Escenario 4 — Ejecución de casos de prueba

Nombre del escenario	ejecutarCasosPrueba
Instancias de actores participantes	luis:Estudiante, motor:MotorCasosDePrueba
Flujo de eventos	1. Luis termina de escribir su solución en la IDE y selecciona la opción "Ejecutar pruebas" en la barra de herramientas. 2. La plataforma valida la integridad criptográfica del archivo antes de enviarlo al motor de pruebas. El hash es consistente. 3. El motor de casos de prueba ejecuta secuencialmente cada caso definido por el profesor, aplicando las entradas configuradas al programa de Luis. 4. Para cada caso, el motor compara la salida del programa con la salida esperada. 5. El motor retorna un reporte: tres casos aprobados y uno fallido. Para el caso fallido, muestra la entrada utilizada, la salida obtenida y la salida esperada. 6. Luis analiza los resultados y decide ajustar su solución para corregir el error detectado.

Table 1.4: Escenario ejecutarCasosPrueba para el caso de uso *Ejecutar Casos de Prueba*.

Escenario 5 — Detección de modificación externa

Nombre del escenario	deteccionModificacionExterna
Instancias de actores participantes	sofia:Estudiante, modCripto:ModuloValidacionCriptografica
Flujo de eventos	1. Sofía intenta abrir un archivo .py de una asignación desde la IDE de la plataforma. 2. El módulo de validación criptográfica calcula el hash actual del archivo y lo compara con la firma registrada previamente. 3. El módulo detecta una discrepancia: el hash calculado no coincide con el hash almacenado, lo que indica que el archivo fue modificado fuera del entorno autorizado. 4. El sistema bloquea el acceso al archivo para edición y muestra un mensaje de advertencia a Sofía, indicando que la integridad del archivo está comprometida. 5. El sistema notifica al profesor sobre el incidente, registrando la fecha, hora y el identificador del archivo afectado. 6. El archivo permanece bloqueado hasta que el profesor intervenga y restablezca su estado de integridad.

Table 1.5: Escenario deteccionModificacionExterna para el caso de uso *Detectar Modificación Externa*.

Escenario 6 — Creación de curso por el profesor

Nombre del escenario	crearCurso
Instancias de actores participantes	prof:Profesor
Flujo de eventos	1. El profesor inicia sesión en la plataforma y accede al panel de administración docente. 2. El profesor selecciona la opción "Crear nuevo curso" e ingresa el nombre, código de curso y descripción del semestre. 3. El sistema solicita al profesor indicar los parámetros de validación criptográfica que se aplicarán a los archivos del curso, tales como el tipo de algoritmo de hash a utilizar. 4. El profesor confirma la configuración y el sistema crea el curso, generando un espacio de trabajo y un repositorio Git asociado. 5. El curso queda disponible para matricular estudiantes y agregar asignaciones.

Table 1.6: Escenario crearCurso para el caso de uso *Gestionar Cursos*.

Escenario 7 — Publicación de asignación con casos de prueba

Nombre del escenario	publicarAsignacion
Instancias de actores participantes	prof:Profesor, motor:MotorCasosDePrueba
Flujo de eventos	1. El profesor accede al curso e ingresa a la sección "Asignaciones" y selecciona "Nueva asignación". 2. El profesor ingresa el título, descripción del ejercicio y la fecha límite de entrega. 3. El profesor define los casos de prueba: para cada caso especifica la entrada estándar (stdin) y la salida esperada (stdout). 4. El motor de casos de prueba valida que el formato de los casos sea correcto y los almacena asociados a la asignación. 5. El profesor publica la asignación. Los estudiantes matriculados en el curso la ven reflejada inmediatamente en su dashboard con el estado "Pendiente".

Table 1.7: Escenario publicarAsignacion para los casos de uso *Gestionar Asignaciones* y *Configurar Casos de Prueba*.

Escenario 8 — Revisión de trabajo estudiantil y retroalimentación

Nombre del escenario	revisarTrabajoEstudiantil
Instancias de actores participantes	prof:Profesor, repGit:RepositorioGit
Flujo de eventos	1. El profesor accede a la lista de entregas de una asignación y selecciona el trabajo de un estudiante en particular. 2. La plataforma consulta al repositorio Git los commits asociados al estudiante para esa asignación y muestra el historial de versiones. 3. El profesor revisa el código fuente del archivo <code>.py</code> en el visor integrado de la plataforma. 4. El profesor agrega comentarios de retroalimentación sobre líneas o secciones específicas del código. 5. Los comentarios quedan registrados en el sistema y el estudiante puede consultarlos al acceder nuevamente a la asignación desde su dashboard.

Table 1.8: Escenario revisarTrabajoEstudiantil para el caso de uso *Revisar Trabajo y Agregar Retroalimentación*.

Escenario 9 — Commit al repositorio Git

Nombre del escenario	realizarCommit
Instancias de actores participantes	diego:Estudiante, repGit:RepositorioGit
Flujo de eventos	1. Diego ha realizado cambios en su archivo <code>.py</code> dentro de la IDE y desea guardar una versión de su progreso. 2. Diego selecciona la opción "Guardar versión" en la IDE. 3. La plataforma valida la integridad criptográfica del archivo y verifica que sea consistente. 4. La plataforma realiza un commit al repositorio Git asociado al curso, con un mensaje automático que incluye la fecha, hora y un resumen de los cambios. 5. El repositorio Git confirma el commit y actualiza el historial de versiones. Diego puede visualizar sus versiones previas desde la interfaz de la plataforma.

Table 1.9: Escenario realizarCommit para el caso de uso *Gestionar Versiones con Git*.

Escenario 10 — Trabajo colaborativo en proyecto grupal

Nombre del escenario	trabajoColaborativo
Instancias de actores participantes	valeria:Estudiante, roberto:Estudiante, repGit:RepositorioGit
Flujo de eventos	1. El profesor ha habilitado un proyecto colaborativo para el grupo integrado por Valeria y Roberto. Ambos lo ven en su dashboard. 2. Valeria accede al proyecto, edita un archivo <code>.py</code> en la IDE y realiza un commit al repositorio compartido del grupo. 3. Roberto accede al proyecto y la plataforma le notifica que existen cambios recientes. Roberto actualiza su espacio de trabajo con los cambios de Valeria. 4. Roberto realiza sus propias modificaciones sobre otros archivos del proyecto y efectúa un nuevo commit. 5. El repositorio Git mantiene el historial diferenciado de commits de Valeria y Roberto, permitiendo al profesor revisar la contribución de cada integrante.

Table 1.10: Escenario trabajoColaborativo para el caso de uso *Colaborar en Proyecto Grupal*.

Chapter 2

Requerimientos Funcionales de Alto Nivel

Los siguientes requerimientos funcionales de alto nivel describen las capacidades que el sistema debe proveer. Cada RF se presenta con su código de seguimiento, título, descripción, prioridad y dependencias funcionales.

Código	RF-01
Título	Autenticación de usuarios
Descripción	El sistema debe permitir que estudiantes y profesores inicien sesión mediante credenciales institucionales. Según el rol detectado, el sistema redirige al usuario a su interfaz correspondiente (dashboard estudiantil o panel docente).
Prioridad	Alta
Dependencias funcionales	Ninguna

Table 2.1: Requerimiento Funcional RF-01.

Código	RF-02
Título	Visualización del dashboard
Descripción	Una vez autenticado, el estudiante debe visualizar su dashboard con la lista de cursos activos y las asignaciones pendientes, indicando título, descripción breve, fecha límite y estado de cada asignación. El profesor visualiza sus cursos y la lista de estudiantes.
Prioridad	Alta
Dependencias funcionales	RF-01

Table 2.2: Requerimiento Funcional RF-02.

Código	RF-03
Título	Gestión de cursos
Descripción	El profesor debe poder crear, editar y eliminar cursos dentro de la plataforma. Al crear un curso, el sistema genera automáticamente un repositorio Git y un espacio de trabajo asociado, y permite al profesor configurar los parámetros iniciales de validación criptográfica.
Prioridad	Alta
Dependencias funcionales	RF-01

Table 2.3: Requerimiento Funcional RF-03.

Código	RF-04
Título	Gestión de asignaciones
Descripción	El profesor debe poder crear, editar y eliminar asignaciones dentro de un curso, especificando título, descripción, fecha límite y si permite trabajo colaborativo. Las asignaciones publicadas deben quedar visibles en el dashboard de los estudiantes matriculados.
Prioridad	Alta
Dependencias funcionales	RF-01, RF-03

Table 2.4: Requerimiento Funcional RF-04.

Código	RF-05
Título	IDE de Python con restricciones
Descripción	La plataforma debe ofrecer una IDE especializada para la creación y edición de archivos <code>.py</code> . Dentro de esta IDE deben estar deshabilitadas las operaciones de copiar y pegar texto, con el objetivo de que el estudiante construya su solución directamente en el entorno controlado.
Prioridad	Alta
Dependencias funcionales	RF-01, RF-02, RF-04

Table 2.5: Requerimiento Funcional RF-05.

Código	RF-06
Título	Validación criptográfica de archivos
Descripción	Cada archivo <code>.py</code> del sistema debe estar asociado a una firma o hash criptográfico que certifique que fue creado y modificado únicamente dentro de la IDE autorizada. El módulo de validación debe calcular y actualizar esta firma en cada guardado legítimo.
Prioridad	Alta
Dependencias funcionales	RF-05

Table 2.6: Requerimiento Funcional RF-06.

Código	RF-07
Título	Detección y bloqueo de modificación externa
Descripción	Cuando el sistema detecte que la firma de un archivo <code>.py</code> no coincide con la registrada, debe bloquear inmediatamente su apertura para edición, notificar al estudiante de la inconsistencia e informar al profesor, registrando el evento con fecha y hora.
Prioridad	Alta
Dependencias funcionales	RF-05, RF-06

Table 2.7: Requerimiento Funcional RF-07.

Código	RF-08
Título	Configuración y ejecución de casos de prueba
Descripción	El profesor debe poder definir casos de prueba para cada asignación, especificando entradas y salidas esperadas. El motor de pruebas debe ejecutar automáticamente cada caso sobre la solución del estudiante y reportar resultados detallados, incluyendo casos aprobados, fallidos y mensajes de error.
Prioridad	Alta
Dependencias funcionales	RF-04, RF-05, RF-06

Table 2.8: Requerimiento Funcional RF-08.

Código	RF-09
Título	Integración con repositorio Git
Descripción	La plataforma debe integrarse con un repositorio Git por curso, permitiendo al estudiante realizar commits desde la IDE sin abandonar el entorno. Cada commit debe incluir metadatos de autoría y marca de tiempo. El profesor debe poder consultar el historial de commits de cada estudiante.
Prioridad	Media
Dependencias funcionales	RF-03, RF-05, RF-06

Table 2.9: Requerimiento Funcional RF-09.

Código	RF-10
Título	Retroalimentación docente
Descripción	El profesor debe poder agregar comentarios de retroalimentación sobre el código de un estudiante, asociados a líneas o bloques específicos del archivo. Los comentarios deben ser visibles para el estudiante al acceder a la asignación desde su dashboard.
Prioridad	Media
Dependencias funcionales	RF-01, RF-03, RF-04, RF-09

Table 2.10: Requerimiento Funcional RF-10.

Código	RF-11
Título	Soporte para trabajo colaborativo
Descripción	El profesor debe poder habilitar proyectos colaborativos para grupos de estudiantes dentro de un curso. Los estudiantes de un mismo grupo deben poder acceder a un espacio de trabajo compartido, realizar commits al repositorio grupal y ver los cambios de sus compañeros.
Prioridad	Media
Dependencias funcionales	RF-04, RF-05, RF-06, RF-09

Table 2.11: Requerimiento Funcional RF-11.

Código	RF-12
Título	Gestión de parámetros criptográficos
Descripción	El profesor debe poder configurar y actualizar los parámetros del mecanismo de validación criptográfica a nivel de curso, tales como el algoritmo de hash utilizado o las llaves públicas asociadas. Los cambios deben propagarse al módulo de validación de forma inmediata.
Prioridad	Alta
Dependencias funcionales	RF-03, RF-06

Table 2.12: Requerimiento Funcional RF-12.

Chapter 3

Diagrama de Casos de Uso

El diagrama de casos de uso de la Figura 3.1 ilustra las interacciones entre los actores del sistema y los casos de uso identificados, empleando notación UML estándar. Las relaciones «include» indican pasos obligatorios y las relaciones «extend» representan comportamientos opcionales o alternativos.

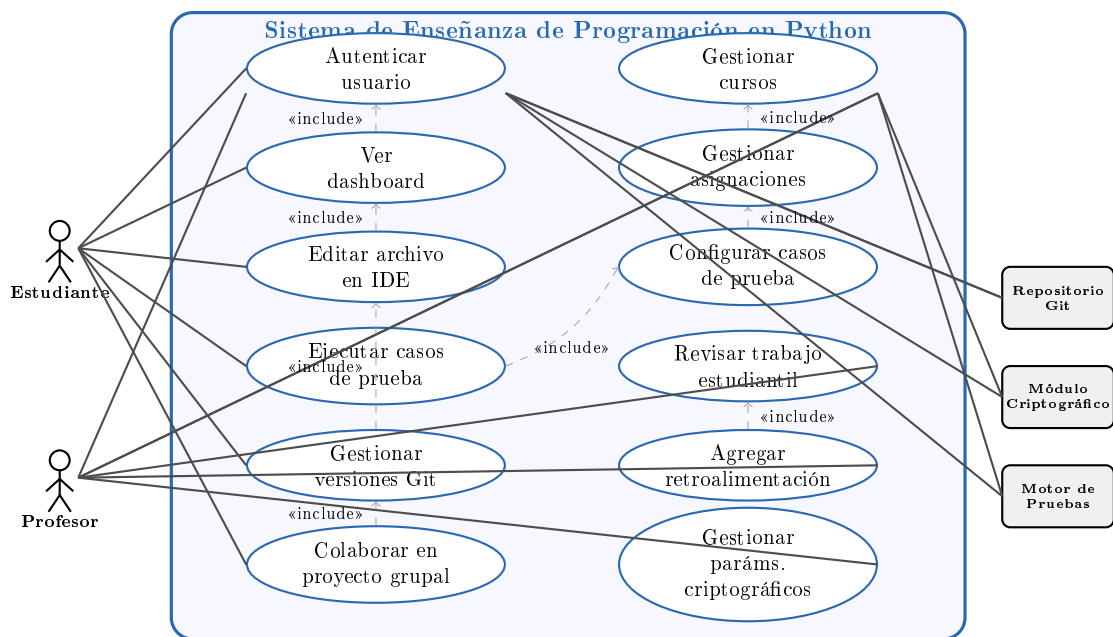


Figure 3.1: Diagrama de casos de uso del Sistema de Enseñanza de Programación en Python.

Chapter 4

Requerimientos No Funcionales (FURPS+)

Los requerimientos no funcionales se organizan siguiendo el esquema **FURPS+**, que agrupa las características de calidad del sistema en: Funcionalidad de seguridad (*Functionality*), Usabilidad (*Usability*), Confiabilidad (*Reliability*), Rendimiento (*Performance*), Soporte (*Supportability*) y restricciones adicionales (+).

4.1 Funcionalidad de seguridad (F)

Código	Nombre	Descripción
RNF-F-01	Control de acceso basado en roles	El sistema debe garantizar que cada usuario acceda únicamente a las funcionalidades correspondientes a su rol (Estudiante o Profesor).
RNF-F-02	Protección de credenciales	Las contraseñas de los usuarios deben almacenarse con algoritmos de <i>hashing</i> seguro (e.g., bcrypt) y nunca en texto plano.
RNF-F-03	Integridad criptográfica obligatoria	Todo archivo <code>.py</code> debe estar firmado criptográficamente antes de poder ser ejecutado o evaluado. Si la firma es inválida, el sistema debe rechazar la operación y registrar el incidente.
RNF-F-04	Trazabilidad de eventos críticos	El sistema debe registrar en un log todos los intentos de modificación externa detectados, incluyendo el identificador del archivo, el estudiante involucrado y la marca de tiempo del evento.

Table 4.1: Requerimientos No Funcionales — Funcionalidad de Seguridad.

4.2 Usabilidad (U)

Código	Nombre	Descripción
RNF-U-01	Accesibilidad de la IDE para principiantes	La IDE debe ofrecer una interfaz simple e intuitiva, adecuada para estudiantes de primer ingreso sin experiencia previa en entornos de desarrollo.
RNF-U-02	Claridad del dashboard	El dashboard debe presentar las asignaciones ordenadas por fecha límite, con indicadores visuales claros del estado de cada una (pendiente, en progreso, entregado).
RNF-U-03	Mensajes de error comprensibles	Los mensajes de error generados por el motor de pruebas y por el módulo de validación criptográfica deben redactarse en lenguaje natural, evitando tecnicismos incomprensibles para estudiantes principiantes.
RNF-U-04	Tiempo de aprendizaje de la plataforma	Un estudiante sin experiencia previa en la plataforma debe ser capaz de acceder, abrir la IDE y realizar su primera entrega en menos de 15 minutos, asistido únicamente por la interfaz del sistema.

Table 4.2: Requerimientos No Funcionales — Usabilidad.

4.3 Confiabilidad (R)

Código	Nombre	Descripción
RNF-R-01	Disponibilidad durante horario académico	El sistema debe garantizar una disponibilidad del 99% durante el horario académico definido (lunes a viernes, 7:00 a.m.–10:00 p.m.).
RNF-R-02	Consistencia del repositorio Git	El repositorio Git no debe perder commits confirmados. En caso de fallo del sistema durante un commit, la operación debe revertirse de forma atómica sin corromper el historial.
RNF-R-03	Persistencia de la validación criptográfica	Las firmas de los archivos deben mantenerse consistentes en el almacenamiento ante reinicios del sistema, sin requerir revalidación masiva.
RNF-R-04	Recuperación ante fallos	En caso de interrupción inesperada durante la edición, la plataforma debe preservar el último guardado válido y restaurar el estado de la sesión al reconectarse.

Table 4.3: Requerimientos No Funcionales — Confiabilidad.

4.4 Rendimiento (P)

Código	Nombre	Descripción
RNF-P-01	Tiempo de ejecución de casos de prueba	El motor de pruebas debe completar la ejecución de todos los casos de una asignación estándar (máximo 10 casos) en menos de 10 segundos para programas Python básicos de primer año.
RNF-P-02	Tiempo de validación criptográfica	La validación criptográfica de un archivo <code>.py</code> no debe representar un retraso perceptible para el usuario; debe completarse en menos de 2 segundos.
RNF-P-03	Capacidad de usuarios concurrentes	La plataforma debe soportar al menos 100 usuarios concurrentes activos (editando código, ejecutando pruebas o realizando commits) sin degradación notable del rendimiento.
RNF-P-04	Carga del dashboard	El dashboard de un estudiante debe cargar completamente en menos de 3 segundos bajo condiciones normales de red y carga del servidor.

Table 4.4: Requerimientos No Funcionales — Rendimiento.

4.5 Soporte (S)

Código	Nombre	Descripción
RNF-S-01	Documentación técnica y de usuario	El sistema debe contar con documentación técnica para desarrolladores y un manual de usuario para estudiantes y profesores, disponibles desde la propia plataforma.
RNF-S-02	Arquitectura modular	El código fuente del sistema debe estar organizado en módulos o componentes desacoplados (IDE, módulo criptográfico, motor de pruebas, integración Git) para facilitar el mantenimiento y la evolución independiente de cada subsistema.
RNF-S-03	Compatibilidad de navegadores	La interfaz web debe funcionar correctamente en las versiones actuales de los principales navegadores: Chrome, Firefox, Edge y Safari.
RNF-S-04	Facilidad de actualización del algoritmo criptográfico	Debe ser posible actualizar el algoritmo de hash utilizado para la validación sin afectar los archivos existentes validados bajo el algoritmo anterior, mediante un proceso de migración documentado.

Table 4.5: Requerimientos No Funcionales — Soporte.

4.6 Restricciones adicionales (+)

4.6.1 Restricciones de diseño

Código	Nombre	Descripción
RNF-D-01	Lenguaje único soportado en la IDE	La IDE debe dar soporte exclusivamente al lenguaje Python, dado que los cursos objetivo son introductorios y basados en dicho lenguaje.
RNF-D-02	Entorno web sin instalación local	La plataforma debe operar completamente desde el navegador, sin requerir que el estudiante instale software adicional en su computadora.

Table 4.6: Requerimientos No Funcionales — Restricciones de Diseño.

4.6.2 Restricciones de implementación

Código	Nombre	Descripción
RNF-I-01	Integración Git mediante protocolos estándar	La integración con el repositorio Git debe realizarse mediante protocolos estándar (HTTPS o SSH), compatibles con servicios como GitHub, GitLab o Gitea.
RNF-I-02	Ejecución de Python en entorno aislado	El motor de casos de prueba debe ejecutar el código del estudiante en un entorno aislado (<i>sandbox</i>) para evitar acceso al sistema de archivos del servidor o al entorno de otros usuarios.

Table 4.7: Requerimientos No Funcionales — Restricciones de Implementación.

4.6.3 Restricciones de interfaz

Código	Nombre	Descripción
RNF-IF-01	Interfaz adaptable a distintas resoluciones	La plataforma debe ser responsiva y adaptarse correctamente a resoluciones de pantalla desde 1024×768 px en adelante.
RNF-IF-02	Sin mensajería interna ni publicación de notas	La plataforma no debe incorporar funcionalidades de mensajería interna entre usuarios ni sistema de publicación de notas finales, ya que estas funciones están fuera del alcance del proyecto.

Table 4.8: Requerimientos No Funcionales — Restricciones de Interfaz.

4.6.4 Restricciones físicas

Código	Nombre	Descripción
RNF-PH-01	Capacidad de almacenamiento de repositorios	El servidor debe contar con capacidad suficiente para almacenar repositorios Git de al menos 50 cursos simultáneos, con un historial de commits de hasta un semestre académico por curso.
RNF-PH-02	Infraestructura institucional	El sistema debe poder desplegarse en la infraestructura de servidores institucionales del TEC, cumpliendo con las políticas de seguridad y administración de recursos definidas por la institución.

Table 4.9: Requerimientos No Funcionales — Restricciones Físicas.